



Universidad Autónoma de la Ciudad de México
Nada humano me es ajeno



Licenciatura en Ingeniería de Software
UACM San Lorenzo Tezonco
introducción a la ingeniería en Software

Practica Final Editor de Texto básico con lista de entrega para generación de archivo

Ariel Sofia Lopez Amaya
Matricula: 21-003-1193

Introducción

En el presente trabajo se desarrolló un programa en lenguaje C que simula un **editor de texto básico**, con la capacidad de crear, modificar y personalizar mensajes dirigidos a un grupo de personas previamente registradas. Esta práctica integra temas fundamentales de la programación en C tales como **arreglos, buffers, punteros, memoria dinámica, estructuras, funciones, y manejo de archivos**, permitiendo así un ejercicio completo e integral de programación estructurada.

El objetivo principal de esta práctica es desarrollar un sistema que permita **combinar un texto base con los datos de una persona (nombre y cargo)** para generar un documento final personalizado, tanto en **pantalla como en archivos de texto**. Este sistema simula la automatización real que se utiliza en ambientes laborales para la creación de oficios, avisos, comunicados y mensajes administrativos personalizados.

Desarrollo

Análisis del Problema

En diferentes áreas administrativas y académicas surge la necesidad de enviar comunicados personalizados a múltiples personas. Hacerlo manualmente implica:

- Repetición de tareas
- Mayor probabilidad de error
- Consumo innecesario de tiempo
- Dificultad para mantener orden y registro

Lo anterior plantea la necesidad de un sistema que:

1. Permita crear y modificar un mensaje base.
2. Capture información de varias personas (nombre y cargo).
3. Combine automáticamente mensaje + persona.
4. Genere un texto personalizado por individuo.
5. Lo muestre en pantalla y lo guarde en un archivo .txt.

Con esto se mejora la productividad y se reduce el trabajo repetitivo.

Solución Propuesta

La solución consiste en un programa en C estructurado de la siguiente manera:

Uso de struct

Representa a las personas mediante:

```
typedef struct {  
    char nombre[50];  
    char cargo[50];  
} Persona;
```

Uso de memoria dinámica

El texto se guarda mediante un puntero:

```
char *texto;  
texto = (char*)malloc(MAX_TEXTO);
```

Funciones modulares

- crearTexto() – captura y almacena el texto base
- modificarTexto() – permite modificar el texto
- capturarPersonas() – guarda nombre y cargo
- generarMensajes() – combina texto + persona y lo guarda en archivos

Uso de un menú con switch

Permite navegar fácilmente entre las opciones.

Generación de archivos individuales

Cada persona recibe su mensaje en un archivo con su nombre:

Sofía.txt

Sofía Lopez.txt

Pruebas Realizadas

Se verificó el correcto funcionamiento:

- Se creó el texto correctamente
- Se modificó sin errores
- Se capturaron varias personas

- Los mensajes personalizados se generaron
- Los archivos .txt se crearon y abrieron sin fallas
- Se validó que la combinación texto-persona fuera correcta

Código

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_TEXTO 500
#define MAX_PERSONAS 20

// ----- STRUCT PARA PERSONAS -----
typedef struct {
    char nombre[50];
    char cargo[50];
} Persona;

// Buffer dinámico para almacenar el texto creado
char *texto = NULL;

// Arreglo de personas
Persona personas[MAX_PERSONAS];
int totalPersonas = 0;

// ----- FUNCIÓN: CREAR TEXTO -----
void crearTexto() {
    texto = (char *)malloc(MAX_TEXTO * sizeof(char));
    if (texto == NULL) {
        printf("Error al asignar memoria.\n");
        return;
    }

    fflush(stdin);
    printf("Escribe el texto base del mensaje:\n> ");
    fgets(texto, MAX_TEXTO, stdin);

    printf("\nTexto guardado correctamente.\n\n");
}

// ----- FUNCIÓN: MODIFICAR TEXTO -----
void modificarTexto() {
    if (texto == NULL) {
```

```

        printf("Primero debes crear un texto.\n\n");
        return;
    }

    fflush(stdin);
    printf("Texto actual:\n%s\n", texto);
    printf("Escribe el nuevo texto:\n> ");
    fgets(texto, MAX_TEXTO, stdin);

    printf("Texto modificado exitosamente.\n\n");
}

// ----- FUNCIÓN: CAPTURAR PERSONAS -----
void capturarPersonas() {
    if (totalPersonas >= MAX_PERSONAS) {
        printf("Se alcanzó el límite máximo de personas.\n");
        return;
    }

    printf("Ingresa el nombre de la persona:\n> ");
    fflush(stdin);
    fgets(personas[totalPersonas].nombre, 50, stdin);

    printf("Ingresa el cargo:\n> ");
    fgets(personas[totalPersonas].cargo, 50, stdin);

    // Eliminar saltos de línea
    personas[totalPersonas].nombre[strcspn(personas[totalPersonas].nombre, "\n")] = 0;
    personas[totalPersonas].cargo[strcspn(personas[totalPersonas].cargo, "\n")] = 0;

    totalPersonas++;
    printf("\nPersona agregada.\n\n");
}

// ----- FUNCIÓN: GENERAR MENSAJES -----
void generarMensajes() {
    if (texto == NULL) {
        printf("Primero debes crear un texto.\n\n");
        return;
    }

    if (totalPersonas == 0) {
        printf("Primero debes capturar personas.\n\n");
        return;
    }

    printf("\n--- MENSAJES PERSONALIZADOS ---\n");
}

```

```

for (int i = 0; i < totalPersonas; i++) {
    printf("\n-----\n");
    printf("Para: %s (%s)\n", personas[i].nombre, personas[i].cargo);
    printf("Mensaje:\n%s\n", texto);

    // Guardar en archivo
    char nombreArchivo[60];
    sprintf(nombreArchivo, "%s.txt", personas[i].nombre);

    FILE *archivo = fopen(nombreArchivo, "w");
    if (archivo != NULL) {
        fprintf(archivo, "Para: %s (%s)\n\n%s", personas[i].nombre, personas[i].cargo,
texto);
        fclose(archivo);
    }
}

printf("\nTodos los archivos fueron generados exitosamente.\n\n");
}

// ----- MENÚ PRINCIPAL -----
int main() {
    int opcion;

    do {
        printf("===== MENU PRINCIPAL =====\n");
        printf("1. Crear texto\n");
        printf("2. Modificar texto\n");
        printf("3. Capturar personas\n");
        printf("4. Generar mensajes personalizados\n");
        printf("5. Salir\n");
        printf("Selecciona una opcion: ");
        scanf("%d", &opcion);
        getchar(); // limpiar buffer

        switch (opcion) {
            case 1: crearTexto(); break;
            case 2: modificarTexto(); break;
            case 3: capturarPersonas(); break;
            case 4: generarMensajes(); break;
            case 5: printf("Saliendo del programa...\n"); break;
            default: printf("Opcion invalida. Intenta de nuevo.\n\n");
        }

    } while (opcion != 5);
}

```

```

// Liberar memoria
if (texto != NULL) free(texto);

return 0;
}

```

Tiempos Estimados y Reales

Actividad	Tiempo Estimado	Tiempo Real
Análisis del problema	20 min	30 min
Diseño del programa	30 min	20 min
Desarrollo del código	60 min	50 min
Pruebas del sistema	30 min	15 min
Redacción del reporte	25 min	30 min
Total	165 min	145 min

Conclusiones

El desarrollo de este programa permitió aplicar de forma integral múltiples conceptos fundamentales del lenguaje C, como el manejo de punteros, memoria dinámica, estructuras, funciones y archivos.

Este ejercicio sirvió no solo para practicar aspectos técnicos, sino también para fortalecer habilidades como el análisis, la abstracción, la resolución de problemas y la capacidad de traducir necesidades reales en soluciones funcionales. Durante el semestre, cada práctica contribuyó progresivamente a comprender cómo organizar un programa, modularizarlo correctamente y depurar errores de manera eficiente.

Este trabajo final refleja el esfuerzo acumulado durante el curso y demuestra el avance logrado en la construcción de programas más completos y útiles. Gracias a la experiencia obtenida, ahora existe una base sólida para continuar desarrollando proyectos más complejos, así como para aplicar estos conocimientos en contextos académicos y profesionales.

```
main.c [ProyectoFinal] - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
<global>
C:\Users\LENOVO\OneDrive\
===== MENU PRINCIPAL =====
1. Crear texto
2. Modificar texto
3. Capturar personas
4. Generar mensajes personalizados
5. Salir
Selecciona una opcion: 1
Escribe el texto base del mensaje:
> Fue una buena materia gracias por enseñar, espero volver a verlo el siguiente semestre

Texto guardado correctamente.

===== MENU PRINCIPAL =====
1. Crear texto
2. Modificar texto
3. Capturar personas
4. Generar mensajes personalizados
5. Salir
Selecciona una opcion: 3
Ingresa el nombre de la persona:
> Raul Jara
Ingresa el cargo:
> Profesor

Persona agregada.

===== MENU PRINCIPAL =====
1. Crear texto
2. Modificar texto
3. Capturar personas
```

```
main.c [ProyectoFinal] - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
<global>
C:\Users\LENOVO\OneDrive\
Ingresa el cargo:
> Profesor

Persona agregada.

===== MENU PRINCIPAL =====
1. Crear texto
2. Modificar texto
3. Capturar personas
4. Generar mensajes personalizados
5. Salir
Selecciona una opcion: 4

--- MENSAJES PERSONALIZADOS ---

Para: Raul Jara (Profesor)
Mensaje:
Fue una buena materia gracias por enseñar, espero volver a verlo el siguiente semestre

Todos los archivos fueron generados exitosamente.

===== MENU PRINCIPAL =====
1. Crear texto
2. Modificar texto
3. Capturar personas
4. Generar mensajes personalizados
5. Salir
Selecciona una opcion: |
```



```
main.c [ProyectoFinal] - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
<global>
Management
Projects
Workspace
ProyectoFinal
Source Files
main.c

Ingresa el cargo:
> Profesor

Persona agregada.

===== MENU PRINCIPAL =====
1. Crear texto
2. Modificar texto
3. Capturar personas
4. Generar mensajes personalizados
5. Salir
Selecciona una opcion: 4

--- MENSAJES PERSONALIZADOS ---

-----
Para: Raul Jara (Profesor)
Mensaje:
Fue una buena materia gracias por enseñar, espero volver a verlo el siguiente semestre

Todos los archivos fueron generados exitosamente.

===== MENU PRINCIPAL =====
1. Crear texto
2. Modificar texto
3. Capturar personas
4. Generar mensajes personalizados
5. Salir
Selecciona una opcion:
```

```
Raul Jara.txt
Archivo Editar Ver
H1
B I
Para: Raul Jara (Profesor)
Fue una buena materia gracias por enseñar, espero volver a verlo el siguiente semestre

Ln 1, Col 1 114 caracteres. Texto sin formato 100% Windows (CRLF) ANSI
01:52 a.m. 22/11/2025
```